

Data Analysis with Python 2024–2025

Parte 3: Mais NumPy Soluções dos Exercícios

Soluções independentes elaboradas para fins de estudo, com base nos enunciados originais

Nota Importante

Este material é uma adaptação das soluções independentes para os exercícios baseados no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**. As soluções abaixo não são o gabarito oficial do curso.

Conteúdo

1 Soluções - Capítulo 3

Exercício 3.11 – para tons de cinza (*to grayscale*)

Solução proposta

```
import numpy as np
import matplotlib.pyplot as plt

def to_grayscale(image):
    # Pesos definidos no problema: r=0.2126, g=0.7152, b=0.0722
    weights = np.array([0.2126, 0.7152, 0.0722])
    # Produto escalar para obter a matriz 2D final em tons de cinza
    return np.dot(image[..., :3], weights)

def to_red(image):
    img = image.copy()
    img[:, :, 1] = 0 # Zera o verde
    img[:, :, 2] = 0 # Zera o azul
    return img

def to_green(image):
    img = image.copy()
    img[:, :, 0] = 0 # Zera o vermelho
    img[:, :, 2] = 0 # Zera o azul
    return img

def to_blue(image):
    img = image.copy()
    img[:, :, 0] = 0 # Zera o vermelho
    img[:, :, 1] = 0 # Zera o verde
    return img

def main():
    painting = plt.imread('src/painting.png')

    gray = to_grayscale(painting)
    plt.gray()
    plt.imshow(gray)
    plt.show()

    fig, ax = plt.subplots(3, 1)
    ax[0].imshow(to_red(painting))
    ax[1].imshow(to_green(painting))
    ax[2].imshow(to_blue(painting))
    plt.show()

if __name__ == '__main__':
    main()
```

Exercício 3.12 – desvanecimento radial (*radial fade*)

Solução proposta

```
import numpy as np
import matplotlib.pyplot as plt

def center(a):
    return (a.shape[0] - 1) / 2.0, (a.shape[1] - 1) / 2.0

def radial_distance(a):
    h, w = a.shape[:2]
    cy, cx = center(a)
    y, x = np.ogrid[0:h, 0:w]
    return np.sqrt((x - cx)**2 + (y - cy)**2)

def scale(a, tmin=0.0, tmax=1.0):
    a_min = np.min(a)
    a_max = np.max(a)
    if a_min == a_max:
        return np.full_like(a, tmin)
    return ((a - a_min) / (a_max - a_min)) * (tmax - tmin) + tmin

def radial_mask(a):
    dist = radial_distance(a)
    scaled_dist = scale(dist)
    return 1 - scaled_dist

def radial_fade(a):
    mask = radial_mask(a)
    # Garante o broadcasting correto com a terceira dimensao (cores)
    return a * mask[..., np.newaxis]

def main():
    painting = plt.imread('src/painting.png')

    fig, ax = plt.subplots(3, 1)
    ax[0].imshow(painting)
    ax[1].imshow(radial_mask(painting), cmap='gray')
    ax[2].imshow(radial_fade(painting))
    plt.show()

if __name__ == '__main__':
    main()
```

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.